

MASKBEAT: LOOPABLE DRUM BEAT GENERATION

Luca A. Lanzendörfer Florian Grötschla Karim Galal Roger Wattenhofer
ETH Zurich

{lanzendoerfer, fgroetschla, kgalal, wattenhofer}@ethz.ch

ABSTRACT

We present MaskBeat, a transformer-based approach for loopable drum pattern generation. Rather than predicting drum hits sequentially, our method uses bidirectional attention with iterative refinement, allowing instruments to be generated in parallel while maintaining musical coherence. Additionally, we introduce custom loss functions that capture drum-specific musical relationships. Our experiments show that MaskBeat generates higher quality and more musically coherent drum patterns than baseline approaches.

1. INTRODUCTION

Generating realistic drum patterns presents several interesting challenges that distinguish it from other music generation tasks. Drums involve complex relationships between instruments, for instance, kick and snare drums typically follow complementary patterns, while closed and open hi-hats are physically mutually exclusive. Different musical genres also have distinct rhythmic signatures, and effective drum patterns need to maintain a consistent groove while still providing musical variation while still providing musical variation [1–8]

Most existing approaches use autoregressive transformers that predict drum hits one instrument at a time [9–14]. However, we observed that this sequential prediction creates problems: Each decision only considers past context, which makes it difficult to maintain global groove consistency. The resulting patterns often suffer from temporal drift and cannot easily be edited or refined.

Our approach, MaskBeat, takes inspiration from MaskGIT [15] and reframes drum generation as a masked token prediction problem. The key insight is that drum patterns benefit from bidirectional context, knowing what happens in future beats can inform better decisions about the current beat. We make several contributions: (1) adapting MaskGIT to drum loops via timestep and instrument masking, (2) introducing groove and dependency losses that embed drum-specific musical knowledge, and (3) run-

ning ablation studies showing that MaskBeat surpasses baselines in musical quality and controllability.

A demo and the codebase of MaskBeat are available at <https://maskbeat.ethz.ch>

2. METHODOLOGY

Token Representation. We represent each timestep as a 9-dimensional binary vector $\mathbf{x}_t \in \{0, 1\}^9$ that encodes which drums are active: [Kick, Snare, Closed Hi-Hat, Open Hi-Hat, Low Tom, Mid Tom, High Tom, Crash, Ride]. Our patterns span 32 steps (2 measures) at 16th-note resolution. This differs from previous work [16] which treats instruments independently.

MaskBeat Architecture. Built on the MaskGIT backbone, our 8-layer, 8-head transformer ($d_{model} = 512$) includes three drum-specific tweaks: (1) *Dual Embedding*: a learned vector for the mask token (id 10) and a linear projection for each 9-D timestep; (2) *Alternating masking*: training begins with full-timestep masks to learn groove, then gradually mixes in per-instrument masks for finer detail; (3) *Fixed 32-step timing signal*: we add a simple repeating sine/cosine pattern so the network can tell "early beat" from "late beat" within every two-bar loop.

Instrument Dependency Loss (DL). In addition to binary cross-entropy, we introduce novel losses that encode musical domain knowledge. Human drummers follow certain musical conventions that our model needs to learn. For example, kick drums typically appear on strong beats (beats 1 and 3), snare drums emphasize backbeats, and it is physically impossible to play both closed and open hi-hat simultaneously. Our dependency loss enforces these relationships: $\mathcal{L}_{dep} = \lambda_{ks} \sum_i [(1 - \hat{y}_{i,kick}) + (1 - \hat{y}_{i+4,snare})] + \lambda_{hh} \sum_t \hat{y}_{t,CHH} \hat{y}_{t,OHH} + \lambda_{tom} \mathcal{L}_{tom}$, where $\hat{y}_{t,instr}$ denotes the model's sigmoid output for a given instrument at timestep t , $\lambda_{ks}=0.15$, $\lambda_{hh}=0.3$ and $\lambda_{tom}=0.1$ weight the kick-snare, hi-hat and tom terms, respectively.

Groove Loss (GL). One key challenge in drum generation is maintaining temporal coherence. Patterns should feel like they have a consistent "pocket" or groove rather than random hits. Our groove loss addresses this by: (1) emphasizing musically important beat positions, and (2) encouraging consistency between similar timepoints across measures: $\mathcal{L}_{groove} = \sum_t w_t \mathcal{L}_{sub}(t) + \beta \sum_m \|\mathbf{P}_m - \mathbf{P}_{m+1}\|^2$, where $w_t \in \{1, 2, 4\}$ emphasises beat importance, $\mathcal{L}_{sub}(t)$ is the squared difference between activations at t and $t-1$, $\mathbf{P}_m$ is the pattern in bar m , and $\beta=0.3$ controls the inter-bar consistency term.



Focal Loss (FL). Drum patterns are naturally sparse, most timestep/instrument combinations are silent (0), with only a few active hits (1). Standard binary cross-entropy treats all predictions equally, but this means the model can achieve low loss by simply predicting mostly zeros everywhere. We counteract this by using a focal loss $\mathcal{L}_{focal} = -\alpha_t(1 - p_t)^\gamma \log p_t$, where p_t is the predicted probability for the true label at timestep t , α_t balances classes and we set $\gamma=2.0$ following [17].

3. EVALUATION & APPLICATION

We construct a dataset from 30k professional drum loops spanning diverse musical genres. We use Groove Monkee¹ which contains 18k loops covering rock, jazz, latin, funk, and world music styles as well as Lakh MIDI [18] with 12k drum tracks from multi-instrument MIDI files. Our dataset covers tempo ranges 60-180 BPM with balanced genre distribution: rock (28%), electronic (22%), funk (18%), jazz (15%), world (17%) [19].

Preprocessing Dataset. We run various stages of pre-processing to ensure high-quality: (1) Swing-preserving quantization, maintains groove feel while normalizing to 16th-note grid using adaptive quantization that preserves swing ratios. (2) Velocity-aware processing, converts MIDI velocities to binary activations using genre-specific thresholds learned from professional recordings [20]. (3) Pattern deduplication, removes near-duplicates using Jaccard similarity with threshold 0.85. (4) Quality filtering, eliminates patterns with excessive density (>40% active cells) or minimal content (<5% active cells). (5) Augmentation strategy, time-shifting (1-16 steps), density adjustment ($\pm 20\%$), and tempo-preserving stretching.

Loop Extraction. To prepare training data from multi-instrument MIDI files we first isolate the drum track (track with `is_drum=True`) and quantize it to a 16th-note grid. We then down-mix the resulting $9 \times T$ roll to a 1-D activation vector $a \in \{0, 1\}^T$ and compute its normalised autocorrelation $\varphi(\tau) = \frac{1}{T} \sum_t a_t a_{t+\tau}$. Searching for lags $\tau \in [16, 64]$, we pick the shortest period τ^* with $\varphi(\tau^*) > 0.8$, a strong indicator of repetition. The contiguous 32-step (two-bar) window starting at that phase is extracted as a loopable drum beat; segments that fail this criterion are discarded. This automatic procedure removes intros/outros and guarantees that every training example can be seamlessly repeated. After additional filters (≥ 6 drum hits, density < 40%) we obtain the 30k loops used for training.

We evaluate different baselines to demonstrate the effectiveness of our proposed losses. As baselines we use an autoregressive transformer [21], MaskGit [15], and MaskGit using our losses. We evaluate using the following metrics: **Beat strength**, which quantifies adherence to musical metrical hierarchy by measuring the ratio of activations on strong beats (1, 3) versus weak beats (2, 4), where higher values indicate more musically structured patterns. **Pattern repetition**, measuring cosine similarity between consecutive 2-measure segments, measuring

Method	Beat Str.	Pat. Rep.	Instr. Bal.
Autoregressive	0.723	0.534	1.87
MaskGIT (MG)	0.758	0.578	2.01
MG + GL	0.789	0.645	2.08
MG + DL	0.821	0.621	2.18
MG + GL + DL	0.834	0.673	2.25
MaskBeat (ours)	0.847	0.692	2.31

Table 1. Ablation study demonstrating the usefulness of the introduced losses. For all metrics, higher means better.

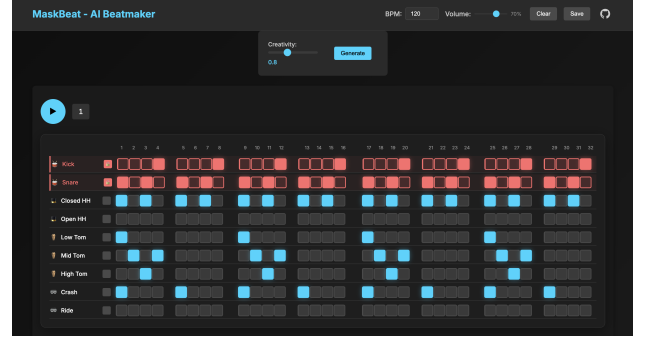


Figure 1. MaskBeat web application interface showing a generated drum pattern. Users can lock elements (shown in red), which are not altered during generation.

groove consistency essential for danceable rhythms (higher means better groove maintenance). **Instrument balance**, measuring Shannon entropy across 9-instrument activation frequencies, capturing rhythmic diversity without bias toward specific drums (higher means more balanced orchestration). Table 1 demonstrates the performance of each model, indicating that the losses introduced in MaskBeat improve model performance.

Novelty. To ensure the model is not memorizing and reproducing the dataset, we compare 1k generated loops against the full training set using token-level intersection-over-union (IoU). No sample exceeded an IoU of 0.90 (exact copy), and the median similarity was 0.31, confirming that MaskBeat produces novel patterns.

Web App. We developed an interactive application (cf. Fig. 1). The interface presents the user with a 9×32 grid representing the drum pattern, where each row corresponds to a drum instrument, and each column represents a 16th-note timestep. Users can lock individual instruments by clicking on instrument rows, which are then not altered during inference. Furthermore, the user can control the sampling temperature using the “creativity” slider. Lower values make the softmax sharper, producing conservative, corpus-like grooves; higher values flatten the distribution and yield riskier, more inventive rhythms.

In summary, we believe MaskBeat represents a step in controllable drum generation by addressing a fundamental challenge: how to create patterns that are both musically coherent and creatively interesting. By incorporating knowledge about drum relationships and groove structure, we achieved significant improvements across all metrics.

¹ <https://groovemonkee.com/>

4. REFERENCES

- [1] J. Gillick, A. Roberts, J. Engel, D. Eck, and D. Baman, "Learning to groove with inverse reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [2] I. Simon, A. Sarroff, and A. Roberts, "Learning a latent space of multitrack measures," 2018.
- [3] X. Liang, J. Wu, and J. Cao, "Midi-sandwich2: Rnn-based hierarchical multi-modal fusion generation vae networks for multi-track symbolic music generation," 2019. [Online]. Available: <https://arxiv.org/abs/1909.03522>
- [4] A. Roberts, J. Engel, C. Raffel, C. Hawthorne, and D. Eck, "A hierarchical latent vector model for learning long-term structure in music," in *International Conference on Machine Learning*, 2018.
- [5] I. Wei, C. Wu, and L. Su, "Generating structured drum patterns using variational autoencoder and self-similarity matrix," in *Proceedings of ISMIR*, 2019, pp. 847–854, symbolic drum-pattern generation with VAE + structural constraints.
- [6] J. Nistal, S. Lattner, and G. Richard, "Drumgan: Synthesis of drum sounds with timbral feature conditioning using generative adversarial networks," *arXiv preprint arXiv:2008.12073*, 2020, audio-domain drum sound generator, not symbolic pattern.
- [7] A. Lavault, A. Roebel, and M. Voiry, "Stylewavegan: Style-based synthesis of drum sounds with extensive controls using generative adversarial networks," *arXiv preprint arXiv:2204.00907*, 2022, audio drum-sound generation, conditioned on drum type and descriptors.
- [8] C. Aouameur, P. Esling, and G. Hadjeres, "Neural drum machine: An interactive system for real-time synthesis of drum sounds," *arXiv preprint arXiv:1907.02637*, 2019, max4Live interface for real-time drum-sound synthesis.
- [9] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, N. Shazeer, I. Simon, C. Hawthorne, A. Dai, M. Hoffman, M. Dinulescu, and D. Eck, "Music transformer: Generating music with long-term structure," in *International Conference on Machine Learning*, 2019, pp. 1364–1372.
- [10] S. Oore, I. Simon, S. Dieleman, D. Eck, and K. Simonyan, "This time with feeling: Learning expressive musical performance," *Neural Computing and Applications*, vol. 32, no. 4, pp. 955–967, 2020.
- [11] A. Wang *et al.*, "Jukedrummer: Conditional beat-aware audio-domain drum accompaniment generation via transformer vq-vae," *ArXiv*, 2023, taiwan AI Labs. [Online]. Available: <https://arxiv.org/abs/2305.12644>
- [12] H. Shah *et al.*, "Generating coherent drum accompaniment with fills and improvisations," *arXiv preprint arXiv:2209.00291*, 2022. [Online]. Available: <https://arxiv.org/pdf/2209.00291>
- [13] P. Jajoria and J. McDermott, "Text conditioned symbolic drumbeat generation using latent diffusion models," 2024. [Online]. Available: <https://arxiv.org/abs/2408.02711>
- [14] V. Shepardson, J. Armitage, and T. Magnusson, "Notochord: a flexible probabilistic model for embodied midi performance," in *Proceedings of the 3rd Conference on AI Music Creativity (AIMC 2022)*, Sep. 2022.
- [15] H. Chang, H. Zhang, L. Jiang, C. Liu, and W. T. Freeman, "Maskgit: Masked generative image transformer," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11 315–11 325.
- [16] H.-W. Dong, W.-Y. Hsiao, L.-C. Yang, and Y.-H. Yang, "Musegan: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment," in *AAAI Conference on Artificial Intelligence*, 2018.
- [17] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2980–2988.
- [18] C. Raffel, *Learning-based methods for comparing sequences, with applications to audio-to-midi alignment and matching*. Columbia University, 2016.
- [19] W.-Y. Hsiao, H.-W. Dong, and Y.-H. Yang, "Compound word transformer: Learning to compose full-song music over dynamic directed hypergraphs," in *AAAI Conference on Artificial Intelligence*, 2021.
- [20] I. Simon *et al.*, "The gigamidi dataset with features for expressive music performance detection," *MetaCreation Research Project*, 2023. [Online]. Available: <https://www.metacreation.net/projects/gigamidi-dataset>
- [21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.